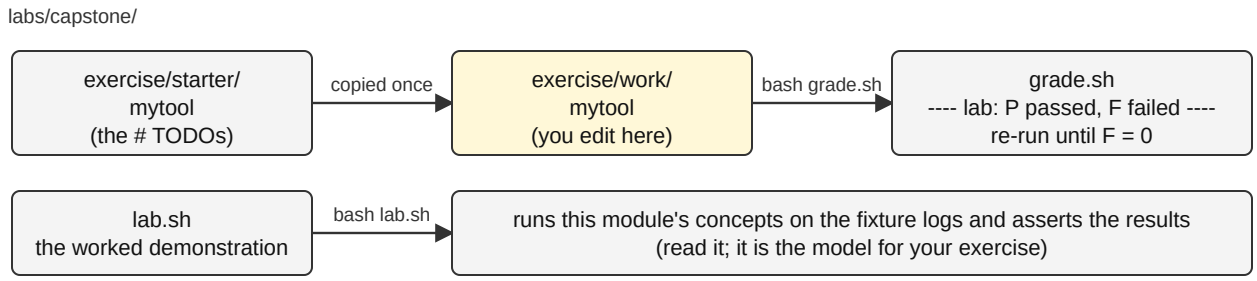
Integrative Practical A-10 — Assemble Your Own Tool: mytool

Topology



Environment Table

All paths are relative to `labs/capstone/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the practical	No
`starter/mytool`	The skeleton of the finished tool with `# TODO` markers in the order to fill them	No — the grader copies it
`work/mytool`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The rubric grader: 12 checks	No
`work/DECISIONS.md`	Your decisions note (≈150 words, three answers)	**Yes** (you create it)
`work/GRADER.txt`	Your final grader output, saved with `bash grade.sh > work/GRADER.txt 2>&1`	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and watch the starter leak a temp file per run.
- **Part 2:** Assemble `mytool`: trap, usage, `getopts`, subcommand check, file-or-stdin, `case` dispatch.
- **Part 3:** Verify with the grader and with `shellcheck`.
- **Part 4:** Write the decisions note.

Background / Scenario

No new concepts. This practical asks you to assemble everything modules M00–M08 taught into one small, real command-line tool, built the way every tool in this course — and every `ci/*.sh` script in the SecMLOps program — is built. `mytool` is a tiny line-analyser:

- `mytool count [FILE]` — the number of lines, from FILE or from stdin when FILE is omitted
- `mytool [-n N] top [FILE]` — the most frequent line(s); `-n N` shows the top N (default 1)
- `mytool -h` — usage on **stdout**, exit 0

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

The standard it must meet: strict mode and a **temp-file cleanup trap** that preserves the exit code (M07); `getopts` for `-n`/`-h` with your own error handling and a `case` dispatcher for the subcommand (M06, M02); input from a **FILE or stdin** (M06); **distinct exit codes** — `0` ok, `1` unreadable file, `2` usage error or unknown command (M06); **shellcheck-clean** (M00, M08).

Time. 60 minutes, self-timed: about 45 to build, 15 for the note. You may use the module pages, any documentation, and AI assistance **for documentation lookup only** — say so in the note. Not allowed: `solution/`, or another learner's `mytool`.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready` (with `shellcheck`).
- Modules M02, M05, M06, M07 (S07.3–S07.4, S06.2, S06.4–S06.5, S02.3, S05.3 are the steps the TODO markers point at).

Part 1: Prepare the exercise folder and watch the leak

Step 1: Run the grader once and count temp files.

```
cd labs/capstone/exercise
bash grade.sh
ls /tmp/mytool.* 2>/dev/null | wc -l
bash grade.sh >/dev/null 2>&1; ls /tmp/mytool.* 2>/dev/null | wc -l
```

Expected: `---- capstone: 2 passed, 10 failed ----`, and a count that grows by one each time you run the grader: the starter creates its temp file but has **no trap**, so every run leaves one behind. You will watch it stop growing once your trap is in. Clean up: `rm -f /tmp/mytool.*`.

Step 2: Read `work/mytool` top to bottom before editing.

The `# TODO` markers are in the order you should fill them: the trap, `usage`, the `getopts` loop, the subcommand check, FILE-or-stdin into the temp file, the `case` dispatch. Each marker says *what* the line must do; the modules showed you *how*.

Part 2: Assemble `mytool`

Work through the markers in order and re-run `bash grade.sh` after each one, reading only the **first** `FAIL` — the checks are ordered so that fixing it usually unlocks the next.

Step 1: The cleanup trap (S07.3–S07.4).

The temp file is already created with `mktemp`. Under the `# shellcheck disable=SC2154` line (keep it), add a trap that captures the exit code first, removes the file, and re-exits with the saved code:

Check it: run the grader twice and `ls /tmp/mytool.*` — the count no longer grows.

Step 2: `usage` (S06.5).

Make `usage` print a short help text to **stdout** — the command line, the two subcommands, the `-n` option, and the FILE-or-stdin rule. A here-document is the natural form.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 3: Option parsing (S06.2).

Parse `-n N` and `-h` with `getopts ":n:h"`: `n` stores the value; `h` prints usage and exits 0; `:` (missing argument) and `\?` (unknown option) print a message to stderr and exit **2**. Then `shift $((OPTIND - 1))`.

Step 4: Require a subcommand (S06.5).

If no argument is left, print usage to **stderr** and exit 2. Otherwise take the first argument as the command and shift it away.

Step 5: Read FILE-or-stdin into the temp file (S06.4).

If a FILE argument remains, check it is readable — if not, print a message to stderr and exit **1** — then copy it into `$tmp` with `cat -- "$1"`. With no FILE, copy stdin into `$tmp`.

Step 6: Dispatch (S02.3, S05.3).

A `case` on `$cmd`: `count` counts the lines of `$tmp`; `top` is the counting pipeline from M05 followed by `sed 's/^ *[0-9]* //'` to drop the counts and keep the lines; anything else prints `unknown command` to stderr and exits **2**.

Step 7: Try it by hand.

```
printf 'apple\nbanana\napple\ncherry\napple\nbanana\n' > /tmp/fruit.txt
bash work/mytool count /tmp/fruit.txt
printf 'a\nb\nc\n' | bash work/mytool count
bash work/mytool -n 2 top /tmp/fruit.txt
bash work/mytool -h | head -1
bash work/mytool frob /tmp/fruit.txt; echo "status=$?"
bash work/mytool count /nope; echo "status=$?"
bash work/mytool -n; echo "status=$?"
rm -f /tmp/fruit.txt
```

Expected: `6`; `3`; `apple` then `banana`; `usage: mytool [-n N] <count|top> [FILE]`; `mytool: unknown command: frob` and `status=2`; `mytool: cannot read file: /nope` and `status=1`; `mytool: -n needs an argument` and `status=2`.

Part 3: Verify

Step 1: shellcheck, then the grader.

```
shellcheck work/mytool
bash grade.sh
bash grade.sh > work/GRADER.txt 2>&1
```

Expected: no shellcheck output, and `---- capstone: 12 passed, 0 failed ----`. The twelve checks, in order: `count` counts lines of a FILE; `count` reads stdin when FILE is omitted; `top` returns the most frequent line; `top -n 2` returns two lines; a good run exits 0; an unknown command exits non-zero; an unknown command exits exactly 2; an unreadable file exits exactly 1; `-h` prints usage and exits 0; runs under strict mode; cleans up with a trap (exactly one line beginning `trap`); shellcheck-clean.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Symptom	Cause	Fix
`-h prints usage and exits 0` fails although `-h` works in your terminal	Usage goes to stderr; the grader runs with `2>/dev/null`	`-h` must print to **stdout**
`unknown command exits exactly 2` fails	The `*)` branch exits 1	Usage mistakes are 2; 1 is for runtime failures
`cleans up with a trap` fails	No `trap` line, or more than one	Exactly one line beginning `trap`
`top` prints counts	The `sed` is missing	`sed 's/^ *[0-9]* //'` after `head`

Step 2: Reset.

`rm -rf work && bash grade.sh`; remove leaked temp files with `rm -f /tmp/mytool.*`.

Part 4: The decisions note

Write `work/DECISIONS.md` — about 150 words in total, in your own words, three short answers. State whether you used AI assistance (documentation lookup only) or none.

D1. Your trap begins with `rc=$?`. Why there, and which of *your* exit codes would be lost without it?

D2. `mytool -h` prints usage to stdout and exits 0; a usage *mistake* prints to stderr and exits 2, not 1. Who depends on each of those two choices?

D3. `mytool` copies its input (file or stdin) into a `mktemp` file before running `wc`/`sort` on it. What does the temp file buy you, and what does the trap then owe in return?

Rubric R-10 and hand-in

Hand in `work/mytool`, `work/DECISIONS.md` and `work/GRADER.txt`. Each criterion is judged **Not met** (0), **Met** (1) or **Strong** (2) and scored `weight × level / 2`. **Pass** = total ≥ 80 % of 20 **and** no *critical* criterion at Not met. *Strong* means "built to the standard the modules taught"; *Met* means "works, with a deviation the modules warned about"; *Not met* means a check fails.

ID	Criterion	Evidence	Weight	Critical
R-10.1	Functional: `count` (file and stdin), `top`, `top -n N`	checks 1–4	5	—
R-10.2	Exit codes are a correct API: 0 ok, 1 unreadable file, 2 usage / unknown command	checks 5–8	3	**yes**
R-10.3	`-h` prints usage to stdout and exits 0	check 9	1	—
R-10.4	Strict-mode header	check 10	1	—
R-10.5	Cleanup trap present and code-preserving	check 11 + the trap line	2	**yes**
R-10.6	shellcheck-clean	check 12	2	**yes**
R-10.7	Decisions note: three answers that name the mechanism in your own words	`DECISIONS.md`	6	—

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

A green grader with no decisions note scores 14/20 = 70 % — **not a pass**. The note is where you show the understanding the grader cannot see.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.